

What we're doing

exp82 is a contact-prediction inference-algorithm harness for the contacts-v1 LM. The question: given a protein's sequence, which decoder extracts the most residue-residue contact signal — `pairwise` scoring, `rollout` frequency-voting, or exp27-style `iterative` growing-K refinement? This PR re-runs that search on our **best** model — the #61/#75 tuned 1.5B (eval loss 2.7566) that #89 exported — and picks + costs the best inference recipe.

The original finding (#67 quick model, eval loss 2.98)

All three methods rank long-range contacts $\sim 2\times$ random but at $\sim 1\text{--}2\%$ absolute precision; `iterative` was marginally best, `rollout` no better than `pairwise`. Verdict: "better inference doesn't rescue a weak base model — the bottleneck is the base model, not the readout." The open follow-up was: re-run on the tuned model.

The open question

#89 took the tuned model (long-range ranking AUC 0.62 → 0.88) but scored ****pairwise only****, assuming exp82's "smarter decoders don't help." That came from the ***weak*** model — yet exp27 got ****+30%** from iteration on a strong base model******. So: now that the base model is strong, does the inference algorithm matter again?

Method selection on a dev set (no test-set hill-climbing)

We **select** the algorithm on a 16-protein ****FoldBench dev**** set (L 100–250, fixed seed-0 sample). Held out for final evaluation: the other 84 FoldBench + 454 denovo/CASP/CAMEO proteins + the entire contacts-v1 test split. Candidate universe = resolved residues; GT = pyconfind contacts ($\text{sep} \geq 6$), identical to #89.

Dev result: rollout > pairwise > iterative (a flip)

long-range R-precision: pairwise 0.16 · **rollout 0.20** · iterative 0.13 · random 0.01.

On the strong model the order inverts vs #67: `rollout` beats `pairwise` on every metric (+20% R), and `iterative` now **hurts** (−18% R). iterative commits the model's own top-K (at ~13–33% precision) as fixed context, seeding mostly *false* contacts that the co-occurrence prior then propagates. `rollout` is variance reduction; its gain is largest at the very top of the ranking.

Sampling sweep: a wash

Sweeping temperature (1.0 / 0.7 / 0.5), top-p, and a domain-aware top-k = $L/5$ moves R-precision by $< \pm 0.006$ — within noise. $T=0.5$ clearly hurts (over-sharpening collapses the vote). Default $T=1.0$ is near-optimal: the lever is the *method*, not the sampling distribution's shape.

Resampling the document per rollout: a small, free bonus

Give each rollout a fresh contacts-v1 realization (resample the N-terminus start + statement order, #89-style TTA) → the vote averages over the document nuisance as well as the sampling noise, at ~no extra GPU cost. +0.007 R on dev, consistent across all 7 metrics. Smaller than #89's +0.05 pairwise TTA — rollout already ensembles. Settled recipe: **rollout + resample**.

Full curated eval (554 proteins): rollout+resample vs pairwise

Long-range, mean over 554: R-precision ****0.285 → 0.368 (+29%)***, contacts@L 0.225 → 0.279, contacts@L/5 0.393 → 0.517. rollout+resample wins every metric on every dataset; the 538 held-out-from-dev proteins give the same (0.289 → 0.373), so it's not dev-overfit. It beats #89's K=10-ensemble **pairwise** (R 0.315) — but stays well below structure methods (ESMFold2 R $\approx$ 0.77).

Cost

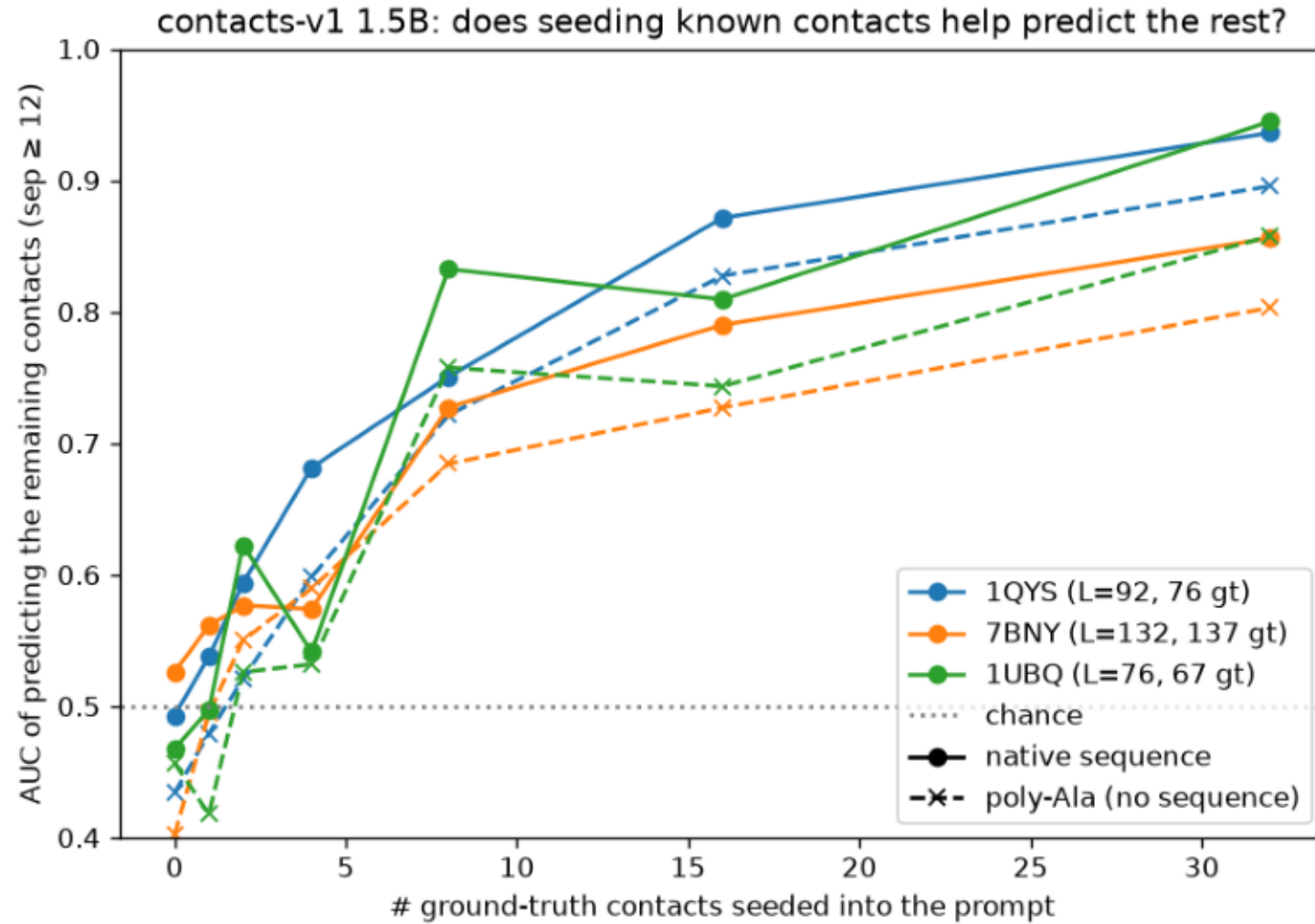
rollout+resample: **~50 s/protein** mean on one A5000 at $n=100$ (median 44 s, max 225 s at $L=738$), ~linear in L ; the full 554 took 8.4 h. pairwise is ~0.3 s. Rollouts terminate cleanly — **0% hit the $4 \cdot L + 64$ cap** — emitting ~2-2.5· L tokens / ~80-125 contacts each. So the +29% R-precision costs ~150× pairwise per protein: cheap in absolute terms (no retraining), not free at scale.

Conclusion

Careful tuning flipped exp82's verdict. On the strong model, **better inference does help**: rollout + per-rollout document-resampling buys **+29% long-range R-precision** over pairwise (what #89 used), for free. `iterative` still hurts (self-seeding needs higher base precision than this model has). The decoder is a real lever again — though closing the gap to structure-based predictors still needs a stronger model, not just a better readout.

auc_vs_seeded.png

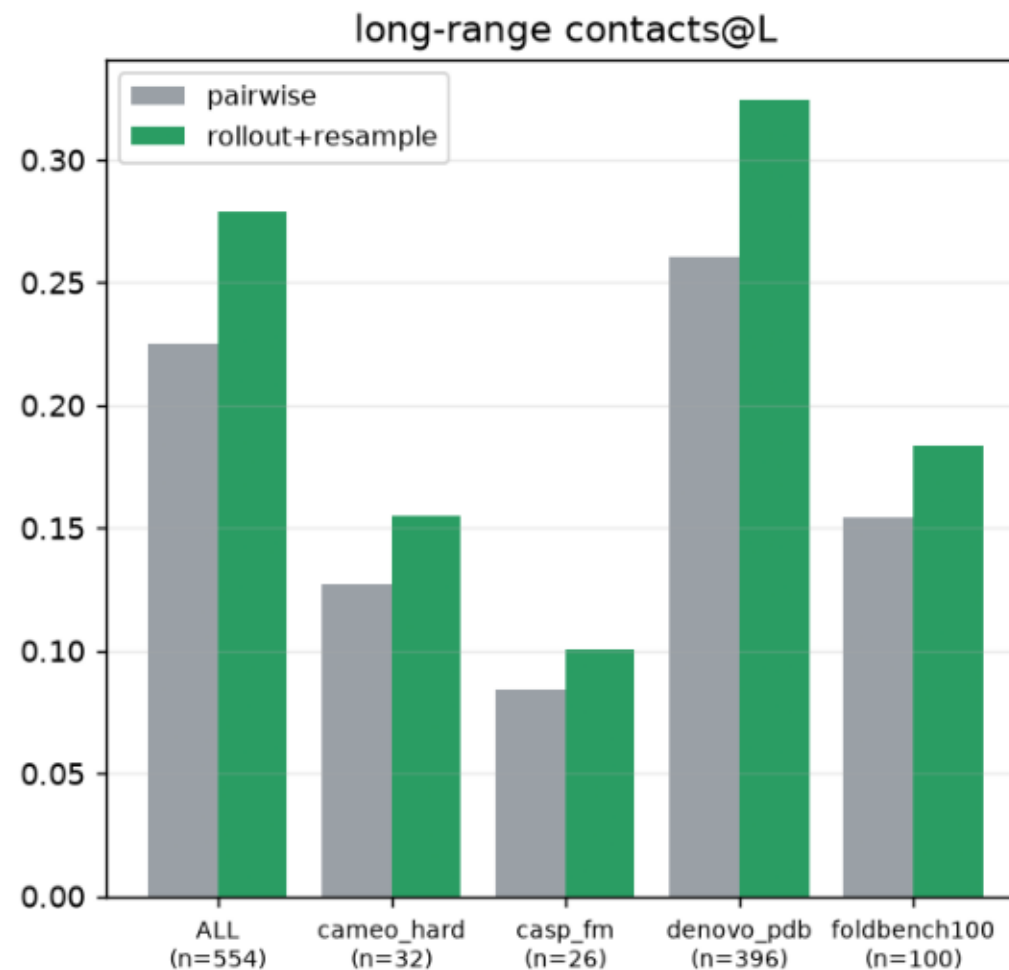
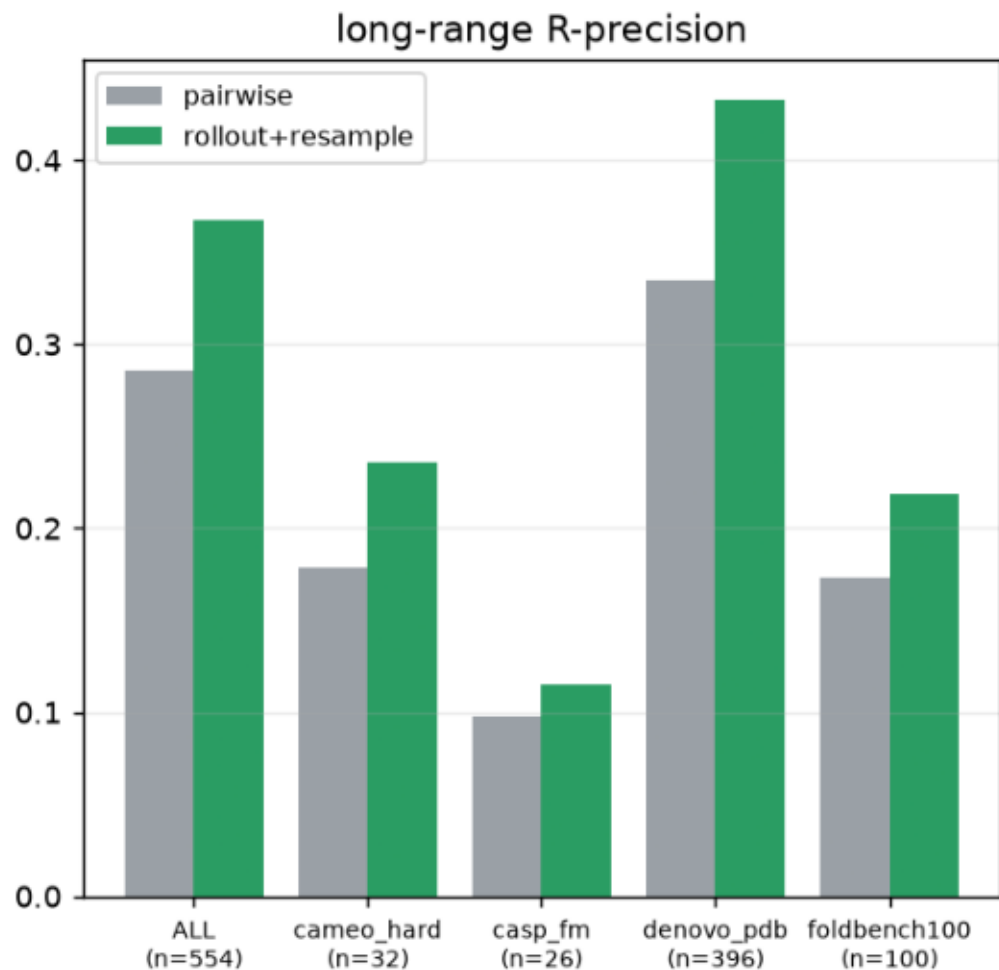
(metadata missing — call `save\_plot\_with\_meta(...)` in the generating script; see build_summary.py)



eval_full_pairwise_vs_resample.png

Long-range (sep $\geq$ 24) contact metrics, pairwise vs the dev-selected rollout+resample recipe, on the 554-protein curated eval set and per dataset. rollout+resample wins every metric on every dataset (see the README table for all bands).

Inference algorithm — pairwise vs rollout+resample (contacts-v1 1.5B, eval loss 2.7566)

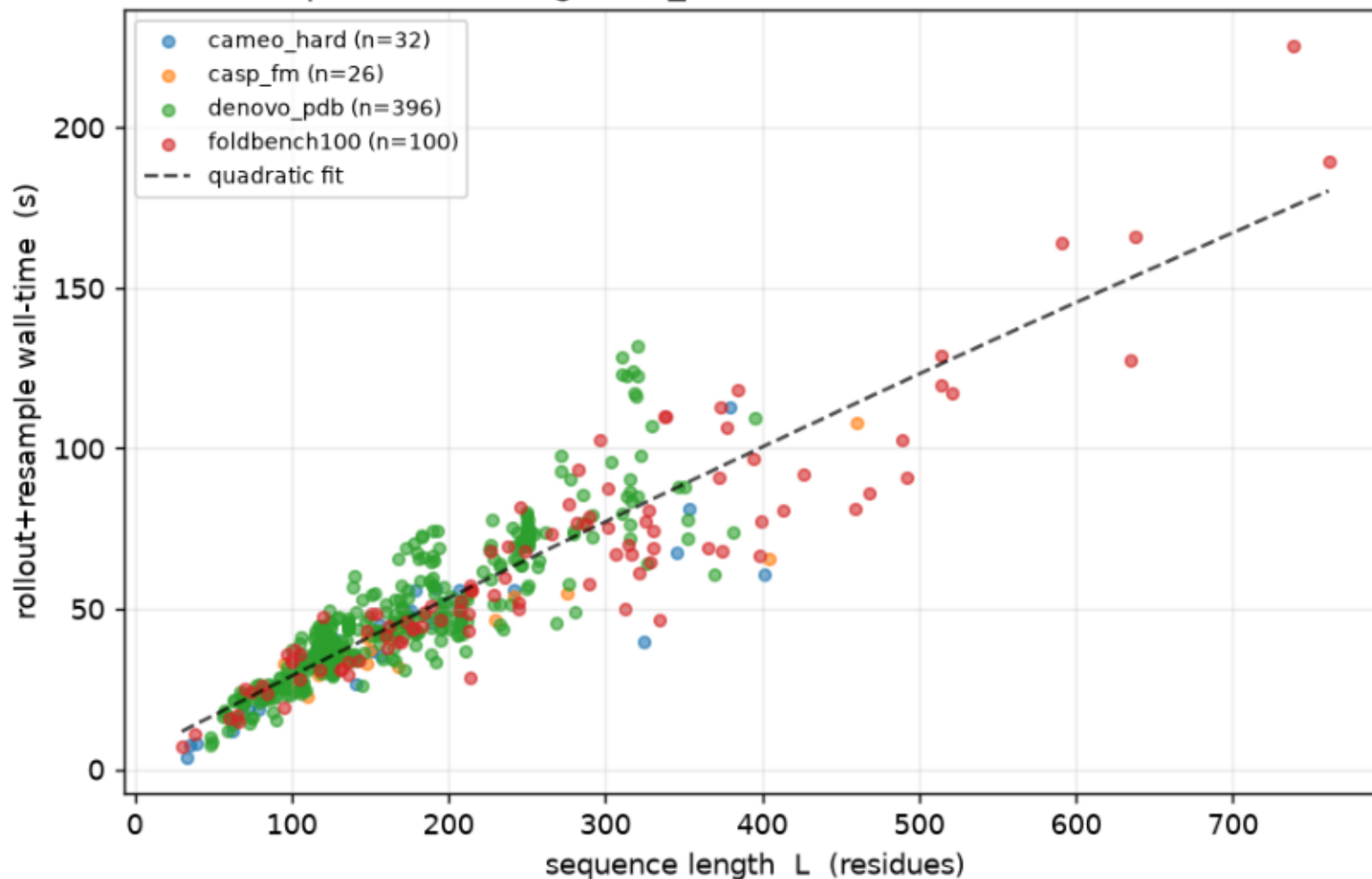


```
$ analyze_full_curated_set.py --results _scratch/eval_full_results.jsonl
```

eval_full_resample_time_vs_length.png

rollout+resample wall-time per protein vs sequence length (one point per protein, 554 curated eval proteins, n_rollouts=100, one A5000). Cost grows ~linearly with L (generation length) with mild super-linear curvature from attention.

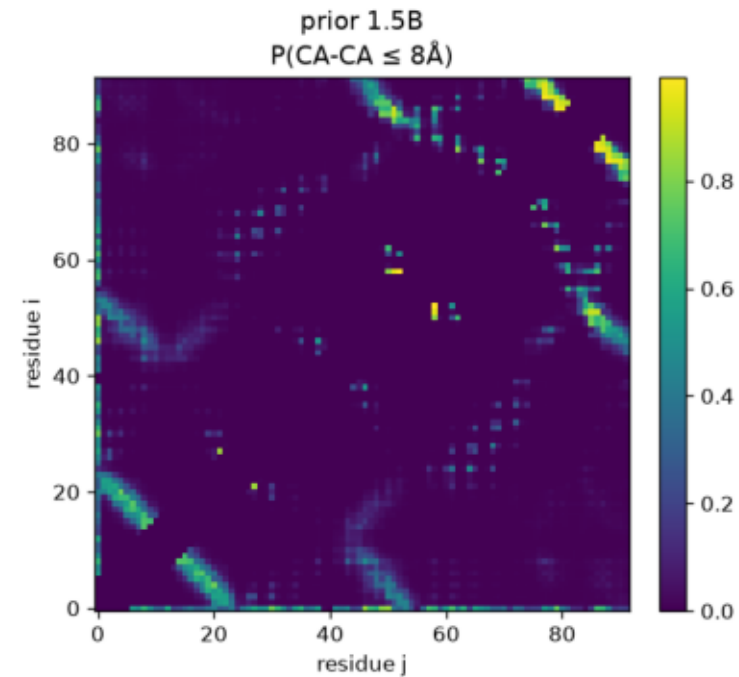
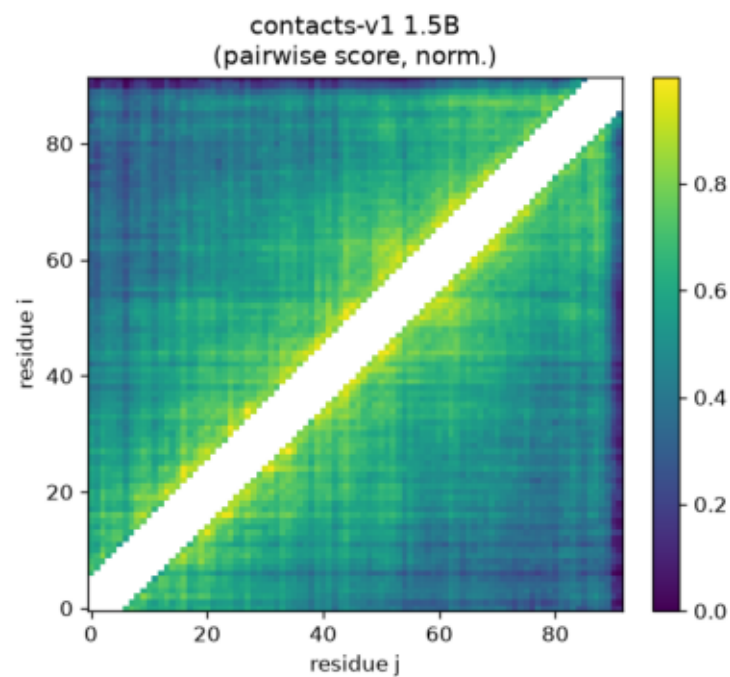
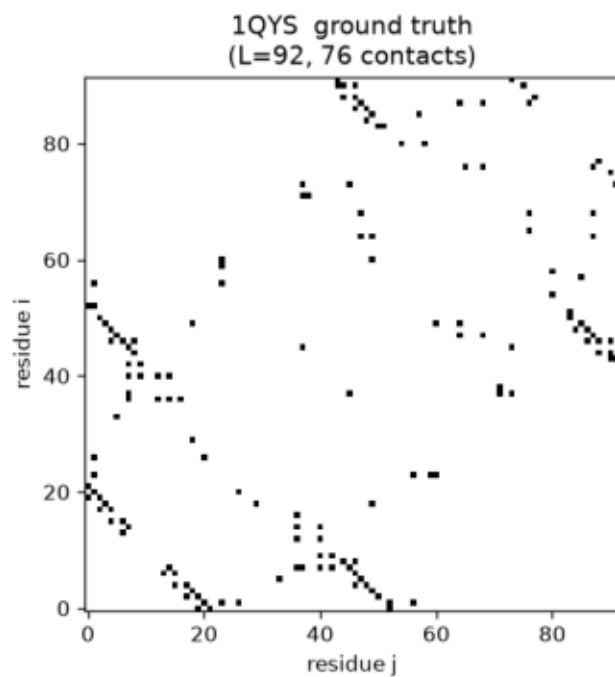
rollout+resample cost vs length (n_rollouts=100, NVIDIA RTX A5000, batch=24)



```
$ analyze_full_curated_set.py --results _scratch/eval_full_results.jsonl
```

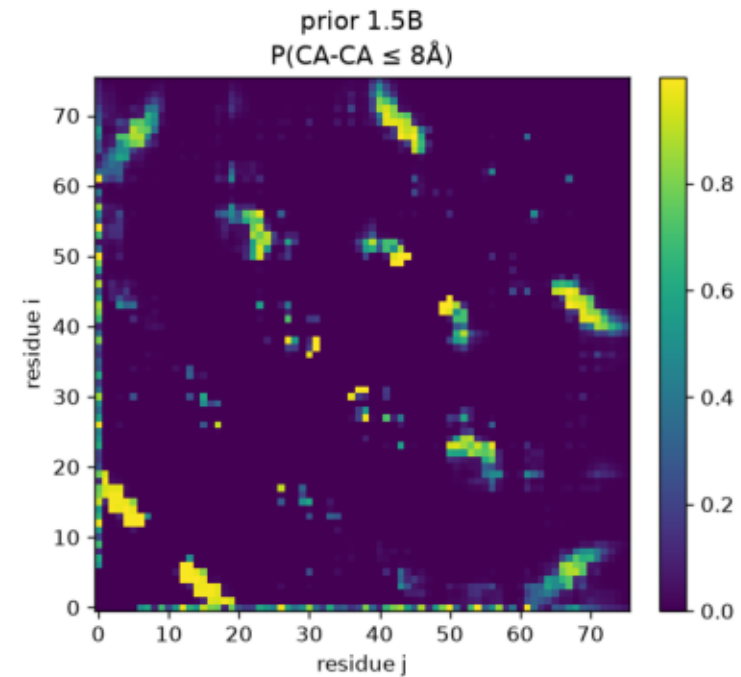
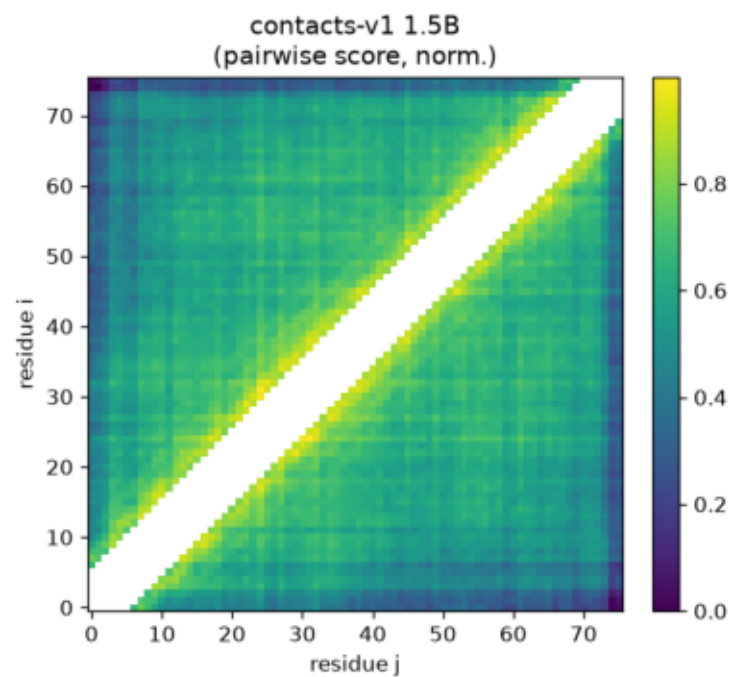
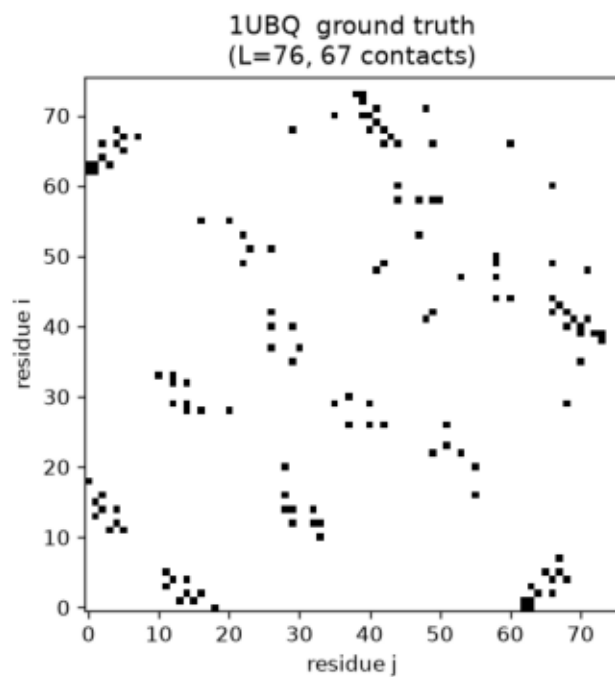
heatmap_1QYS.png

(metadata missing — call ``save_plot_with_meta(...)`` in the generating script; see `build_summary.py`)



heatmap_1UBQ.png

(metadata missing — call ``save_plot_with_meta(...)`` in the generating script; see `build_summary.py`)



heatmap_7BNY.png

(metadata missing — call `save\_plot\_with\_meta(...)` in the generating script; see build_summary.py)

